

Overview

Prompt engineering is the practice of creating the proper prompt to generate the output that you want. With the proper prompting, an LLM can do an amazing number of tasks, including generating an email or product description, summarizing provided text, classifying text into standard or customized categories, responding to a customer query with an appropriate answer, and much more.

Every model behaves differently given the same prompt, and so you'll probably spend a fair amount of time adjusting your prompt for your specific use case or adapting it for different models. The best practices given here are not absolute rules, but guidelines.

Components of a prompt

A prompt, in our context, consists of the following information. Other than the instruction, all other elements depend on the task and improving or customizing the response.

- Instruction: What the model should do, often in great detail.
- The format or syntax of the response (Markdown/text, HTML, JSON, XML).
- The persona of the LLM ("a friendly travel agent").
- Any stylistic or other guidelines for the output ("simple, plain language, no more than two sentences long")
- Examples to follow, if appropriate.

- Any text to be analyzed or acted upon
(such as a user's question or financial or medical information)
-

Developing the prompt

Your first prompt is rarely good enough, particularly when designing a prompt to use for a commercial system. You'll spend a lot of time refining your prompt and assessing the results.

Here is a typical workflow for designing and refining a prompt:

1. Start with just the instruction, without examples. You can ask the LLM for help creating a starter prompt and modify it from there.
 2. Test your prompt against one example, and evaluate the quality.
 3. Modify the prompt, test, and repeat.
 - While you're adjusting your prompt, set the temperature to 0 to get consistent answers, then gradually increase the temperature in small increments up to where you are getting the amount of variation that you want.
 - When the model produces bad output, bring the temperature back to zero and adjust the model again to try to understand what's producing the bad output.
 4. Collect a test set of 10 common inputs and ideal outputs and test your prompt against those inputs.
 - Be sure that your test prompts include both common inputs and inputs that test the extremes of valid requests.
 - Include test examples of bad input to see how the model reacts to it.
 5. Grade the test results against the expected result (you can grade on a score from 1-10 or 1-100). Evaluate the results, adjust your prompt, and repeat.
-

Prompt design tips

Be concise

Say everything you want to say with as few words as possible. Don't state the obvious.



Bad

You are a customer support representative for ACME corp, your name is Wile E. Coyote. you need to answer user questions regarding support issues. Be polite, engaging and to the point.

Do not curse.

Do not mention competitors.



Better

You are Wile E., ACME corp support chat representative.

Be polite, engaging and to the point.

Ensure that the prompt is clear

When crafting prompts for Jamba models, follow this fundamental principle: Write your prompts as you would want them to be written for you. If you can't understand the prompt well, neither will an LLM.

To elaborate:

- **Clarity and Comprehensibility:** Ensure that your prompt is free from grammatical errors and that the instructions are expressed clearly and unambiguously.
- **Formatting and Examples:** Having a structured output allows further manipulation after generation.



Bad

List all the following animals, objects and places in the story.



Better

List all the following animals, objects and places in the story.

Story:

Your lists:

Your output should be in the following format:

```
{  
  "animals": ["all animals in the world"],  
  "objects": ["all objects in the world"],  
  "places": ["all places in the world"]  
}
```

Story: story

– End of story –

Clean and well-structured prompts minimize errors and help you debug and optimize your instructions. If you encounter difficulties or errors that you can't seem to fix, try simplifying your prompt.

Describe the DOs, not the DON'Ts

Focus on telling the model what you want to do. Minimize "do nots." Of course, you can use negatives occasionally, but excessive use of "don't do" and "avoid x" is a sign that your **prompt may be going in the wrong direction** and you may want to rewrite your prompt.



Bad

Write a product description for a high-end cell phone (i.e. not a landline). The description should not be for regular folks; it should only be for important executives. Do not make it overly sales-ish; instead have it be grounded in the specs of the phone.



Better

Write a product description for a high-end cell phone. The description should be tailored for a high powered executive and focus on the specs of the phone. Focus on how the phone can enable more efficient work.

Allow the model to say "I don't know"

Specifically state to the model you permit it to not return an answer. This reduces hallucinations.



Bad

Why did revenue increase,
according to the following quarterly
report?

Quarterly Report



Better

Why did revenue increase,
according to the following quarterly
report? If the answer is not in the
provided report, reply only with "I
don't know"

Quarterly Report

System prompt - Use it!

Describe the role that the LLM should assume when answering the question. This is frequently referred to as a *system prompt*, and has been found to produce better completions for many families of LLMs. This affects not only the tone and language used, but also the amount of detail and level of expertise used.

System prompts should also be used to guide the perspective the model takes when answering the question; for example, by thinking about the problem as a research assistant, or a customer, or a novice.

When accessing the model in code, the system prompt is specified by an initial `role:system` message. In the playground, provide the information in the *System instructions* section. Alternatively, you can put the system prompt directly into the prompt itself, although this might be less effective.



Bad

I want you to assume the role of a
meticulous research assistant.



Better

Your task is to Evaluate if the following text extract is relevant to the case at hand.

You are a meticulous, critical research assistant.

In the SDK

```
def single_message_instruct():
    prompt = """ Evaluate if the following text extract is relevant to the cas
Extract:
...
Case:
...
"""

    messages = [
        ChatMessage(
            role="system",
            content="You are a meticulous research assistant"
        ),
        ChatMessage(
            role="user",
            content=prompt
        )
    ]
    response = client.chat.completions.create(
        model="jamba-instruct",
        messages=messages,
        temperature=0.7
    )
    print(response.choices[0].message.content)
```

The IDH Template (Instruction—Data—Hint)

For complex prompts, include instructions, then data, then a hint. This can be used for simple prompts as well. The hint should be a paraphrased version of the instructions.



Bad



Better

Rewrite the following patient record, so that it is easily understandable for an average person with a high school degree.

{PATIENT RECORD}

Rewrite the following patient record, so that it is easily understandable for an average person with a high school degree.

{PATIENT RECORD}

high school level rewrite:

For prompts with a lot of data it is better to clearly state to the model where every section starts and ends.



Bad

Your task is to fix the product description to be compliant with the product guidance.

{Product Description}

{Product Guidance}

Your product description:



Better

Your task is to fix the product description to be compliant with the product guidance.

Original product description:

{Product Description} – End of Description –

Product Guidance: {Product Guidance} – End of Description –

Your product description:

For some prompts with complex instructions, it is useful to include instruction both in the beginning and in the end of the prompt.



Bad

Your task is to fix the product description to be compliant with the



Better

Your task is to fix the product description to be compliant with the

product guidance.

Original product description:

{Product Description}

– End of Description –

Product Guidance:

{VERY COMPLEX Product
Guidance}

– End of Description –

Your product description:

product guidance.

Original product description:

{Product Description}

– End of Description –

Product Guidance:

{VERY COMPLEX Product
Guidance}

– End of Description –

The rewrite product description in
accordance with the product
guidance:

Use structured output when needed If your output is meant to be read by another system (e.g., for integration into a pipeline), request a JSON-formatted response. Use the `response_format=json` API parameter and specify the expected structure in the prompt itself.

Maintaining output quality in production

Log your prompts and responses in production. Periodically check the output quality for a random selection of your generated answers. Provide a feedback button that sends you the question and generated response to help you improve your prompts.

Use structured output when needed

If your output is meant to be read by another system (e.g., for integration into a pipeline), request a JSON-formatted response.

Use the `response_format=json` API parameter and specify the expected structure in the prompt itself.



Bad



Better

Extract the user's name, location, and request from the input text.

Extract the user's name, location, and request from the input text. Return the output in the following JSON format:

```
{ "name": "", "location": "",  
  "request": "" }
```

Use the appropriate tool

For straightforward math calculations or other actions that can be done in simple code, use a more appropriate tool for the job (a calculator, a macro, a short code snippet). Those tools are designed specifically for the job, and provide much more controllable and consistent output than an LLM.

Ask the LLM if it understands the prompt

To speed up development, consider first asking the LLM if it understands the instructions and other key terms in the prompt. This can help ensure that the LLM understands the core idea of what you are trying to do.

Don't do math

LLMs are famously bad at math. They're getting better, but it's still not a good idea, generally to ask an LLM to solve a word problem for you. LLMs can count and do sums, but word problems or logic are tricky and produce inconsistent results. For simple or straightforward math, use a calculator or other more appropriate tool. If you must evaluate a word problem or logic, you can increase the accuracy by asking the LLM to explain each step it takes in the process (called *chain-of-thought prompting*).

Classification/Ranking

When performing classification or other scoring, it is much better to use categories that are meaningful rather than arbitrary numbers.



Bad

Analyze whether the two sentences provided are consistent or



Better

Analyze whether the two sentences provided are consistent or

inconsistent. You can provide a score of between 1-3.

Sentence A:

"When talking on the phone, the defendant confessed to felony murder."

Sentence B:

"The defendant admitted nothing when talking on the phone."

Your score:

inconsistent. Classify into one of the following classes: [Consistent, Partially Consistent, Inconsistent]

Sentence A:

"When talking on the phone, the defendant confessed to felony murder."

Sentence B:

"The defendant admitted nothing when talking on the phone."

Your classification:

Consistency vs. creativity

Output variability is influenced by two parameters: *temperature_and_top P*:

- *Temperature* Flattens or enhances the curve of probability values for all candidates in the pool. When the curve is completely flat ($T=\max$) there is an equal likelihood for any candidate in the pool to be chosen, no matter how unlikely the candidate originally was. When the curve is completely enhanced ($T=0$), the most likely candidate is now the only choice, and all the other candidates have probability zero. Temperature does not adjust the size of the pool by eliminating candidates, unless you choose $T=0$, which essentially removes all candidates from the pool except the most likely. For most use cases, a temperature of around 0.7 is about right.
- *Top P* controls the size of the pool of candidates for the next token (remember P for Pool). The smaller the candidate pool, the less variability. Candidates are cut in order of least likely to most likely, so at its smallest size (0.01), only the most likely candidates—possibly only one—will be in the pool. At 1.0, the pool is full sized and contains all possible candidates (including some possible weirdos). In practice you won't need to adjust the top P often.

If you need completely consistent answers, such as for classification or math (but don't do math), set the temperature to 0. When you want some variation, start with a low temperature (0.2-0.3) and increase by tenths until you get the variability that you want. Typically you won't need a temperature higher than 0.7. Note that setting temperatures higher than 0.7 can cause the LLM to wander, and setting it higher than 1.0 can cause extremely long and sometimes nonsensical output (definitely specify a `max_tokens` limit for high temperatures). If the temperature is very high (greater than 0.7) reduce the top P a few tenths of a point to remove the very unlikely results unless you want some very high creativity, in which case you can keep the topP high (try a topP of 0.99 to omit extremes).

Limiting output length

If you have a length goal or limit for your output, specify it in the prompt as the desired or maximum number of lines, words, sentences, or paragraphs. Don't expect the model to hit the mark exactly, as it can only see one word ahead, and it might need a bit more or less than you specify to provide a good answer.

Examples: "Write two or three sentences about..." or "Limit your answer to 10 words." The API supports a `max_tokens` parameter, but you should use it only as a failsafe to prevent the edge case of the model going off in a completely unexpected direction and far exceeding your length limits (higher temperature can increase output length). This value is absolutely respected, so if the output hits this limit, the result might stop in the middle of a word. Set this value a fair bit higher than any prompt-suggested limit.

Requesting citations

LLMs have been known to invent citations, so asking Jamba Instruct for citations for its information is not a guarantee of accuracy. If you need absolutely reliable citations, use the RAG Engine.

Use labels, not numbers, to rate output

It is common for people to want a numerical scoring of "good" or "bad" ("on a scale of 1 to 10..."). Assigning exact numbers to subjective categories is hard for people, and harder for LLMs, and also gives a false sense of accuracy. Simple labels like "Bad," "Okay," and "Best" are easier for the language model to provide than precise numerical ratings like 4.7 or 6.8.

❗ Although you can use numbers to represent categories it is generally preferable to use category labels that are inherently meaningful, such as “None”, “Some”, and “Most”.

I want you to analyze whether the two sentences provided are consistent or inconsistent. You can provide a score of None, Some, Most, where “Most” means the most inconsistent.

#####

Sentence A: Josh went to the store and then went to school

Sentence B: Josh never went to school

Output: Most

#####

Sentence A: Josh has a dog and a cat

Sentence B: Josh has a fish

Output: None

####

Sentence A: Josh prefers typing on his laptop

Sentence B: Josh prefers typing on his phone and his laptop

Output: Some

####

Sentence A: When talking on the phone, the defendant confessed to felony murder

Sentence B: The defendant admitted nothing when talking on the phone.

Providing examples

Providing examples of what you want to see (also called “Few Shot prompts”) can be very useful to Jamba. Examples can be very helpful when

1. Providing clear instruction is laborious/difficult, or
2. The model seems to not fully understand your instructions.

Before you provide examples, try out the results using just instruction to see if you get the results you need. If it turns out that the results look better with an example or two, or if describing something is harder than showing an example, then go ahead and use examples. Some recommendations about using examples in your prompt:

- Separate examples clearly with a blank line, or another obvious marker (some people use ###).

- Put the instructions all at the beginning or at the end.
- Be sure all examples use the same structure; don't show different examples with different fields, or totally different structure.
- When providing just a few examples, the ordering of examples probably doesn't matter. When using the RAG engine, or providing very, very large examples or input, you might find that the order of files used as input might affect the output (a "recency bias").

In the following example, we use two types of delimiters: a delimiter `###` between examples and a newline between ads and answers. We've put the examples first and the instructions at the end. Content between `user[" "]` marks are just placeholders in the example where you would put actual ads, answers, or criteria.

Examples:

EXAMPLE AD 1

EXAMPLE AD 2

EXAMPLE ANSWER 1

####

EXAMPLE AD 3

EXAMPLE AD 4

EXAMPLE ANSWER 2

###

I want you to decide which of the following two advertisements are best, given

Criteria:

- Shorter is generally better
- If there is some kind of rhyme within the ad, that can add a little value
- References to superheroes are a huge plus

...

AD 1

AD 2

Cover all cases in your examples

When providing examples, do your best to cover all relevant example types. For example, if the answer to a question posed to the LLM can be “yes” or “no,” provide examples where the answer is yes and examples where the answer is no. Ideally, the distribution should match real-life use cases as well. In the following example, we provide prompt examples that show how to respond to both vague and specific feedback from the user.

```
For the following negative review of a product, suggest a way the comment can
product change if the review is negative. If the review contains nothing spec
message asking for clarification for what is wrong.
```

```
####
```

```
Input:
```

```
I really didn't like your blender, it was defective.
```

```
Output:
```

```
Hi! We really appreciate your feedback about the blender. Can you elaborate
was defective? We are always improving our blenders and really would like to
```

```
####
```

```
Input:
```

```
I recently bought your top of the line laptop with 64K memory, but it ran out
```

```
Output:
```

```
We should consider increasing the hard drive storage to be significantly large
```

```
####
```

```
Input: I recently bought a car from this company and was extremely disappointed
it broke down multiple times within the first week. The car had a number of
have been fixed before being sold.
```

```
Output:
```

Grading results

During development, and also after release (if you're using your prompt in a production system) you'll need to evaluate your results. Which methods you use depend on your usage scale and where you are in the development process. For grading outputs with an absolute answer (such as classification or sentiment analysis), judge on the following criteria:

- Accuracy (how many times the answer was correct)

- For prompts that are sensitive to wrong answers, calculate false positives and false negatives, and the cost of each, when deciding the bar to reach before deciding that your prompt is good enough.

Create an ideal answer for each test input and grade the generated result against the golden answer on a scale of 1-10. This is especially useful for classification exercises, which you will mark as correct or incorrect.

Human evaluation

If you have the resources, human evaluation is often the best solution. Typically in the early stages of adjusting your prompts you'll be evaluating the answers yourself. Create a list of criteria to consider when evaluating each answer (accuracy, clarity, usefulness). You can either rate each criteria individually or give a general overall score (1-5, good/OK/bad).

Use an LLM to evaluate your results

You can try using another LLM to rank your answers. If you do, use a different LLM than the one that generated the answer (LLMs, like people, tend to be biased toward their own answers). For example, if you have a large block of information and your prompt asks the model to answer a specific question based on that information, you might write a prompt like this to evaluate the result generated from your first prompt:

<<context from original prompt>>

###

Based on the context above, does the following answer provide a correct and

Pay attention to the following criteria:

- The answer must be correct according to the information in the context
- The answer that uses information that is not available in the context is a
- If the question can't be answered by the information in the context, the a
- If the answer is in the context, and is provided, that should increase the
- The answer must fully answer the question and not be too short.
- All relevant information should be in the answer.
- A vague or general answer is not very good.

Reply in the following format:

- Score: Good, OK, or Bad
- Reasoning: Why this score was chosen.

###

<<answer from original prompt>>

Maintaining output quality in production

Log your prompts and responses in production. Periodically check the output quality for a random selection of your generated answers. Provide a feedback button that sends you the question and generated response to help you improve your prompts.

Break up complex requests into multiple prompts

If you have a complex task that requires several steps, you might want to break it into multiple prompts, and port the output of each step into the prompt for the next step. That way you can fine tune the results (and temperature) for each step. For example, to provide a list of doctors for a patient with a specific symptom, you might break it into these steps:

1. Given the patient's symptoms, determine what the likely problem is.
2. Given the problem, decide which type of doctor handles that type of problem.
3. Look up the list of doctors with that specialty and provide contact information for each.

Break long inputs into smaller segments

When asking the model to process or rewrite very long documents (above 10k tokens), don't submit them in a single pass. The model is likely to compress, summarize, or truncate the content. Instead, explicitly instruct it to divide the source into smaller, logically coherent segments (around 400–800 words each), aligned with natural boundaries like headings or topic shifts. Process each segment in order, preserving its detail and length, then stitch the outputs back together with light editing for flow.



Bad

Rewrite the following 12,000-word document while keeping the same level of detail and length.



Better

Divide the following document into coherent sections of about 400–800 words. For each section, rewrite it with the same level of detail and approximately the same length, keeping the order intact. After processing all sections, combine the rewritten segments into a single continuous text.

Was this page helpful?

 Yes

 No